Context & Harness Engineering

Lessons from AKD

Nishan Pantha · NASA-IMPACT

2nd ESA-NASA International Workshop on AI Foundation Models for Earth Observation

Day 4 / Track 1 — GeoAI Agent Tutorial · May 2026

The bottleneck has moved

> *"Context engineering is the delicate art and science of filling the context window with just the right information for the next step."*
> — ***Andrej Karpathy**, June 2025*

The **model** isn't the bottleneck anymore.

The **system around the model** is.

PROMPT → CONTEXT CONTEXT → HARNESS

Prompt engineering → context engineering

Prompt engineering → context engineering

a string becomes a system — what changes when you scale to scientific agents

PROMPT ENGINEERING

a clever string

artefact

one prompt template; you tweak words.

unit of work

a single LLM call.

scaling

"add another paragraph."

failure mode

wrong wording → wrong answer.

evaluation

vibes / spot-check.

CONTEXT ENGINEERING

a system that curates

artefact

a directory of files + a tool surface + guardrails.

unit of work

a run: write → select → compress → isolate, then call.

scaling

retrieve / compact / partition across sub-agents.

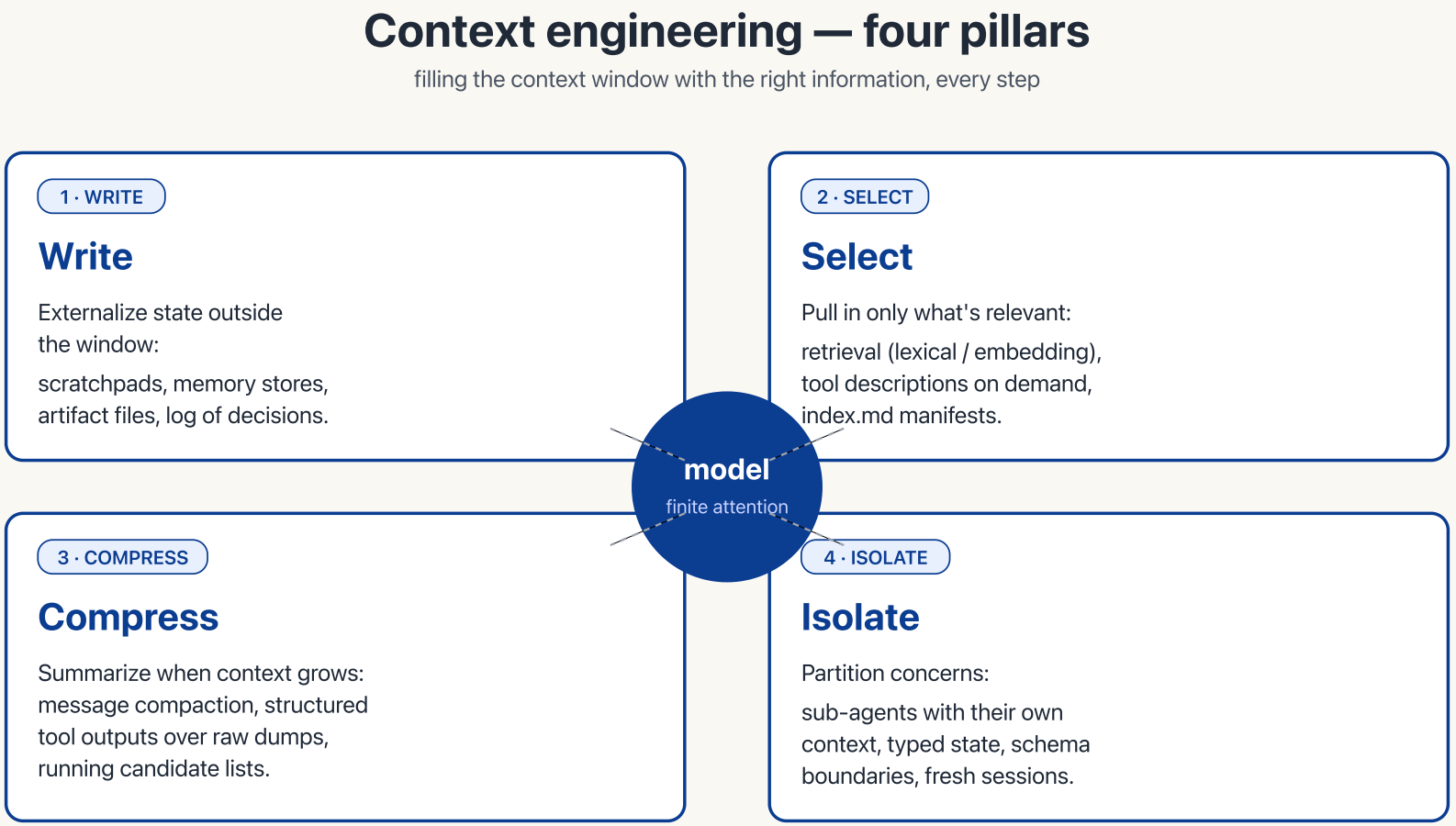
failure mode

poisoning · distraction · confusion · clash · context rot.

evaluation

benchmark suites · trajectory evals · reproducibility logs.

The four pillars



attributed to LangChain — the canonical decomposition

How contexts fail

Bigger window ≠ better answers

four ways contexts fail in production

01 Poisoning

A hallucination, bad tool output, or stale fact lands in context — then propagates downstream as if it were ground truth.

02 Distraction

Window so full of "relevant" history that the model loses sight of what the user actually asked. Attention budget gone.

03 Confusion

Irrelevant tool outputs, dead references, or stray instructions muddle the next decision. Signal drowned by noise.

04 Clash

Two pieces of context disagree. System prompt says one thing, retrieved doc the other. The model picks — sometimes wrong.

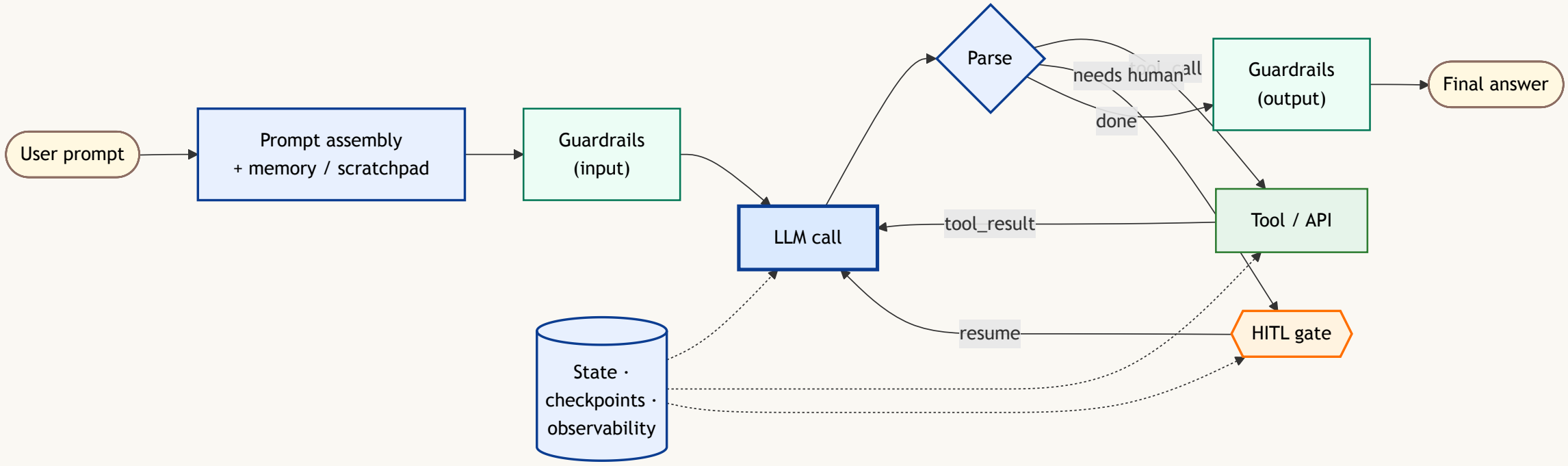
Context rot

Chroma (2025): every frontier model degrades as input grows. Long ≠ thoughtful.
Treat the window as a finite attention budget, not free storage.

refs: Modarressi et al. "NoLiMa" arXiv:2502.05167 (2025) · Chroma "Context Rot" (2025) · dbreunig.com "How Long Contexts Fail" (2025)

Harness engineering

> ***"~98% of Claude Code is deterministic infrastructure."*** *— the agent loop is a while-loop; the harness does the work.*



A capable model + a poor harness loses to a weaker model + a great harness. Reliability lives in the surrounding **deterministic** code — tools, loop, guardrails, state.

The live architectural debate

Cognition (Devin)

"Don't build multi-agents."

One thread, one context.
Actions carry implicit decisions; multiple agents make conflicting decisions.

→ single-agent + long context + compaction.

Anthropic (Research)

Multi-agent works — if you isolate.

Orchestrator + sub-agents, each with own context, explicit hand-offs.

→ parallelism for breadth; isolation prevents clash.

AKD's pragmatic middle: single thread *per agent*; multi-agent only at workflow boundaries with explicit hand-offs and human approval gates.

What others are doing — and what AKD borrows

Coding & general agents

- **Claude Code** — deterministic harness · `CLAUDE.md` / **Agent Skills** · permissions
- **Devin / Cognition** — single thread · long context · checkpointing
- **AutoGen · Magentic-One** (MS) — planner + multi-agent orchestrator
- **OpenAI Deep Research** — plan → retrieve → synthesize

Scientific agents

- **Sakana AI Scientist v2** — end-to-end paper generation
- **Google AI co-scientist** — hypothesis generation + critique
- **ChemCrow** — chemistry tool-using agent
- **MS × NASA hydrology** · NASA deep-research bots — EO domain

AKD inherits — deterministic harness + HITL (*Claude Code*) · single thread per agent (*Devin*) · LangGraph at workflow edges (*AutoGen*) · plan→retrieve→synthesize (*Deep Research*) · tool curation + benchmarks (*ChemCrow, AI Scientist*).

Meet AKD

Accelerated Knowledge Discovery — NASA-IMPACT program.

A **composable multi-agent framework** for NASA's Science Mission Directorate.

Five divisions

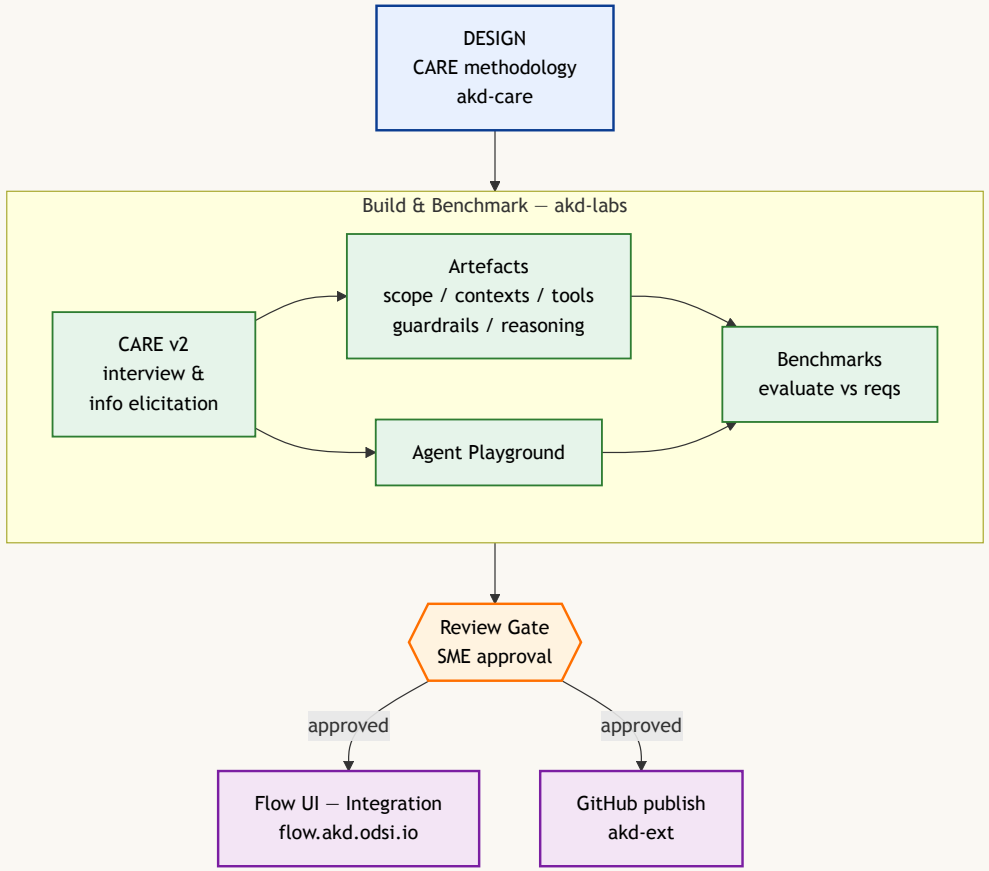
- Earth Science
- Planetary Science
- Astrophysics
- Heliophysics
- Biological & Physical Sciences

Five working repos

- `akd-core` — primitives
- `akd-ext` — domain agents + tools
- `akd-services` — Flow **SDK** · agent · workflow · DB substrate for any project
- `akd-flow` — Next.js frontend
- `akd-labs` — full **lifecycle**: design (CARE) · build · debug · synth-bench

Today — 6 domain agents · the CM1 pipeline · 2 guardrail providers · live at **flow.akd.odsi.io**.

Agent development lifecycle



Every published agent passes through this loop. **Context engineering is built in** — not a step at the end.

Context engineering @ AKD — CARE

CARE = Collaborative Agent Reasoning Engineering

A staged, artefact-driven discipline. Five phases:

#	Phase	Output
1	**Scope**	Who, what, when, success criteria
2	**Key-info elicitation**	Domain knowledge interview transcripts
3	**Reasoning policy**	The loop the agent will run
4	**Prompt architecture**	The artifact directory
5	**Benchmarking**	Test suite + rubric

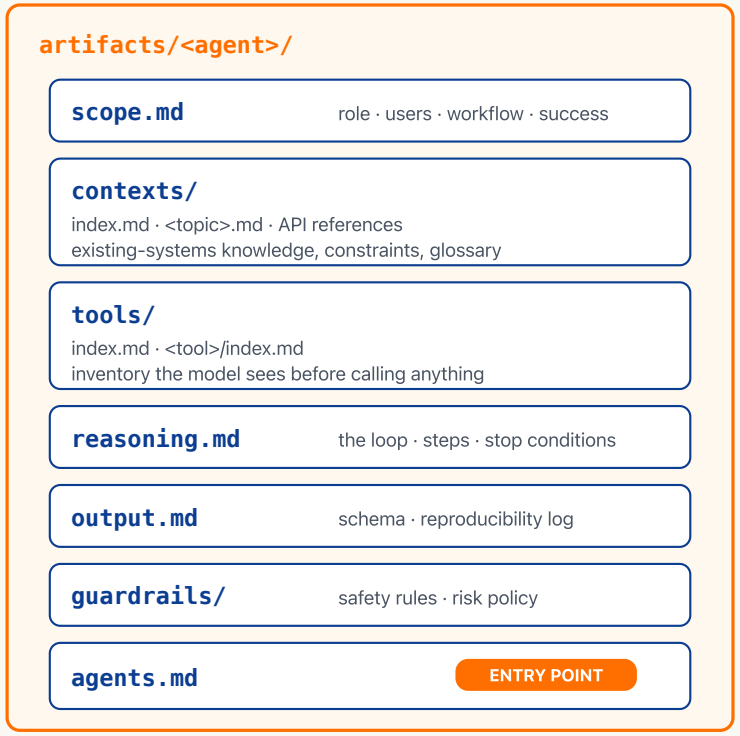
> *Context engineering — but as a discipline, not an afterthought.*

→ akd-care repository · **lineage:** tool-curation rigour from *ChemCrow* · prompt architecture from *Claude Code's CLAUDE.md / AGENTS.md*

Artifacts as the knowledge layer

Artifact-driven context — AKD & the tutorial agent share this shape

behavior is written, not coded · each file is consumable knowledge for the LLM



Why this shape works

WRITE
behavior lives in files, not embedded Python strings — diff-able, reviewable

SELECT
agent reads `agents.md` first, then pulls in specific files on demand via console tools

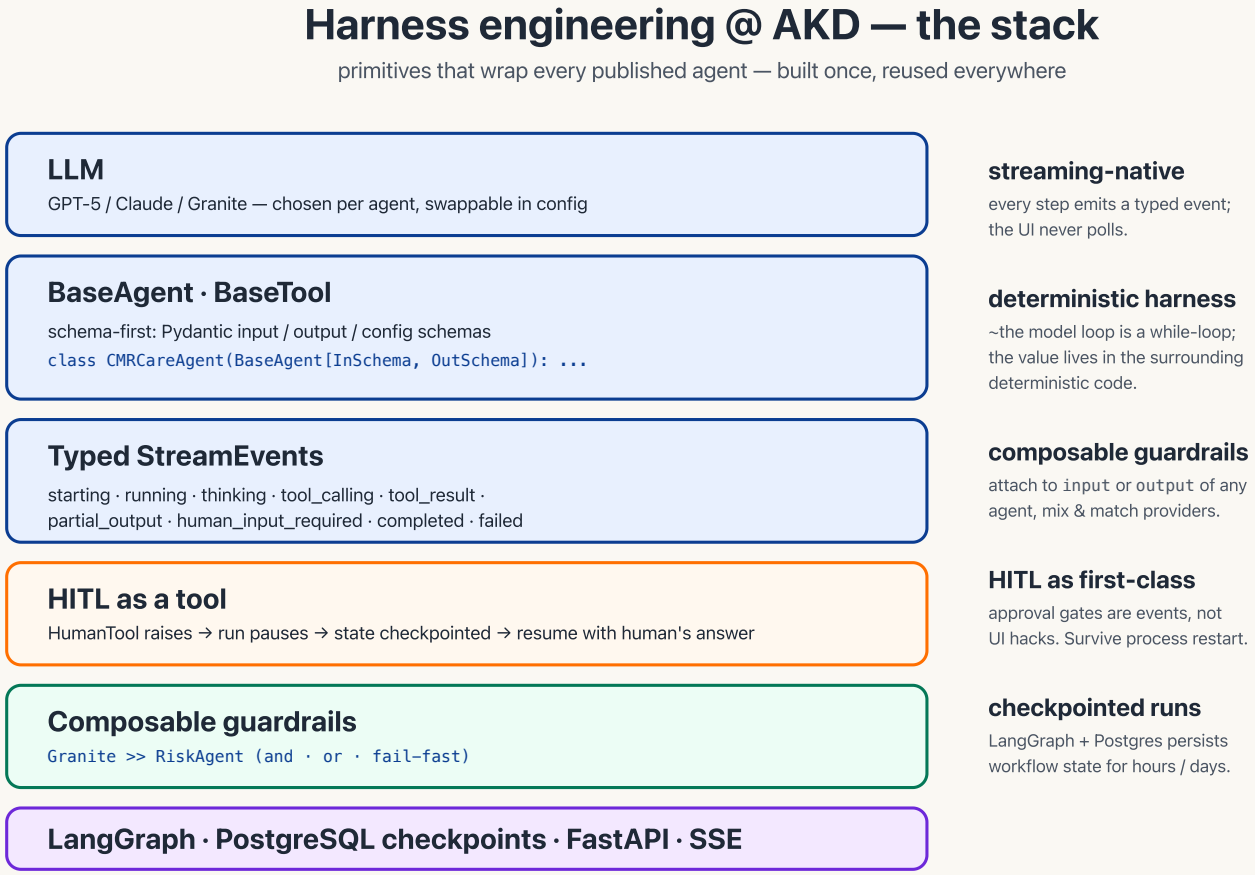
COMPRESS
`index.md` manifests describe each dir — no `grep`, no blanket `ls`; targeted reads

ISOLATE
each agent has its own artifact tree — no shared mutable state

AKD README · line 88
"Artifact content is code-free. The per-agent artifacts/ directories are written as the consumable knowledge layer for agents."

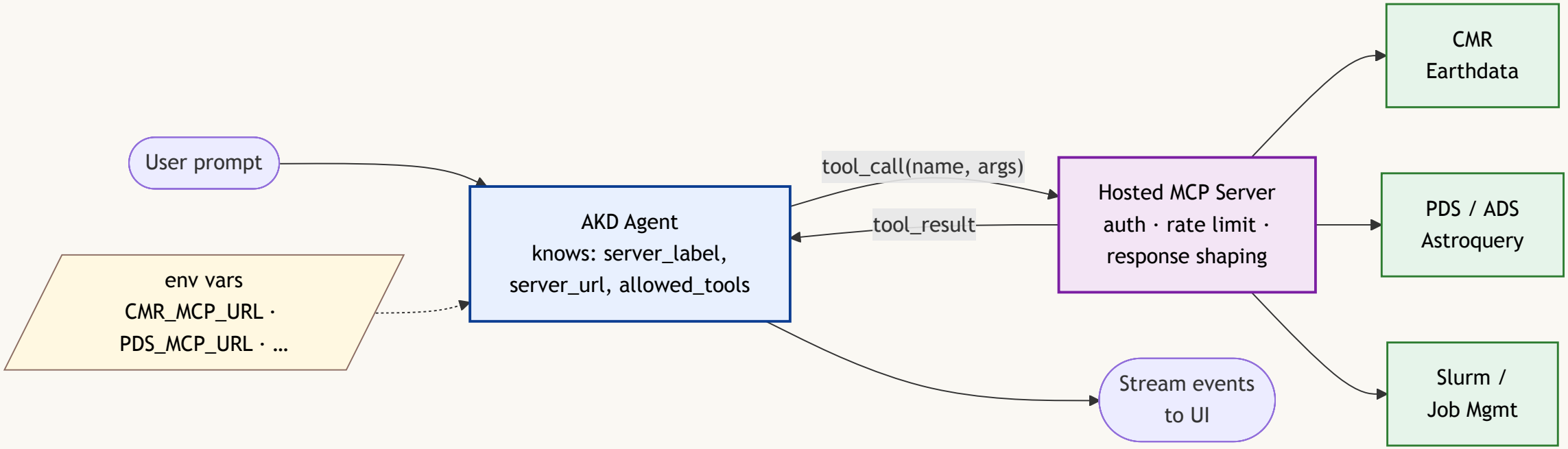
lineage: evolution of *Claude Code's* `CLAUDE.md` + *Anthropic's* **Agent Skills** (`SKILL.md` + supporting files); AKD calls them **artifacts** — the *tutorial agent* uses this pattern verbatim.

Harness engineering @ AKD



lineage: typed-events + checkpointing from *LangGraph* · HITL-as-tool from *Claude Code's* permission model · guardrail composition extends *OpenAI Agents SDK* patterns.

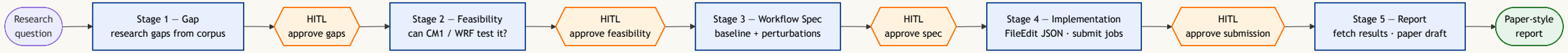
MCP — pluggable scientific data



Agent code never touches API keys or HTTP clients. Swap an upstream source by changing one env var. Same MCP server feeds multiple agents — `allowed_tools` differs.

lineage: *Anthropic's* Model Context Protocol, now adopted by OpenAI Agents SDK · AKD applies it to **NASA data systems**

End-to-end: Closed-Loop CM1



Atmospheric simulation research with **CM1 / WRF**. Five autonomous stages, **human approval between every stage**.

Each stage has its own artifact, context, and guardrails. Hand-offs are explicit.

lineage: staged-pipeline pattern from *Sakana AI Scientist* + *Google AI co-scientist*, with HITL gates between stages (AKD addition).

Takeaways — and the tie-back

Three things to take away

1. **Artifact-driven context** — write the agent's knowledge as files, not code.
2. **Deterministic harness** — the loop is small; the surrounding infra is most of the value.
3. **Evaluation closes the loop** — without benchmarking, "context engineering" is just vibes.

Tie-back to today's tutorial

The Prithvi agent you'll run loads from an **artifact** with `scope.md / contexts/ / tools/ / guardrails/ / reasoning.md / output.md / agents.md`.

Same shape as AKD. Same ideas, applied to EO foundation models.

Read: akd-suite repo · Anthropic *Effective context engineering* · Cognition *Don't build multi-agents* · dbreunig.com *How Long Contexts Fail* · Chroma *Context Rot* · OpenAI *Harness engineering*